

Project 2

Automated Quality Inspection (Computer Vision)

Using OpenCV · Python · Image Processing · Logic Gates

Complete Implementation Guide

Table of Contents

Section	Topic
1	Project Overview & Objectives
2	System Architecture & Pipeline
3	Environment Setup
4	Step-by-Step Code Implementation
5	Defect Detection Logic & Pass/Fail Criteria
6	Running the System
7	Sample Output & Results
8	Conclusion & Extensions

1. Project Overview & Objectives

This project involves building an **Automated Optical Inspection (AOI) system** using Python and OpenCV. The system monitors a conveyor belt via a live camera feed or image dataset and automatically classifies each part as **PASS** or **FAIL** based on visual defects such as surface cracks, missing components, or dimensional errors.

1.1 Key Requirements

Requirement	Details
Input	Live video feed (webcam/IP camera) or image dataset folder
Processing	Grayscale conversion → Gaussian Blur → Thresholding → Contour detection
Output	Bounding boxes drawn on defects; PASS / FAIL label per part
Skills Used	OpenCV, Python, Image Processing, Logic Gates (Pass/Fail criteria)
Defects	Surface crack (contour isolation), Missing component (region analysis)

2. System Architecture & Pipeline

The inspection pipeline is a linear sequence of image-processing stages. Each stage transforms the image and feeds it to the next:

Stage	Operation	Purpose
1	Image Capture	Grab frame from webcam / load from disk
2	Grayscale	Convert BGR → single-channel gray (reduces complexity)
3	Gaussian Blur	Smooth image to suppress sensor noise
4	Thresholding	Binarise: pixels become black (0) or white (255)
5	Contour Detection	Find connected regions (potential defects)
6	Bounding Box	Draw rectangles around each suspect region
7	Pass / Fail Logic	Area / count thresholds → classification label
8	Display / Log	Show result on screen and write to CSV log

■ *Design Principle: Each stage is a pure function — it takes an image/data in and returns a transformed image/data out. This makes the pipeline easy to debug and extend with new processing stages.*

3. Environment Setup

3.1 Prerequisites

Install the following before writing any code:

```
# Create a virtual environment (recommended)

python -m venv venv

source venv/bin/activate # Linux / macOS
venv\Scripts\activate # Windows

# Install required packages

pip install opencv-python numpy matplotlib

# Optional – for image datasets / logging

pip install Pillow pandas
```

3.2 Project Folder Structure

```
project2_quality_inspection/
  ■■■ main.py ← Entry point (webcam or image mode)
  ■■■ pipeline.py ← Image processing functions
  ■■■ defect_logic.py ← Pass/Fail threshold logic
  ■■■ utils.py ← Helpers (logging, file I/O)
  ■■■ images/
  ■ ■■■ good_parts/ ← Sample PASS images
  ■ ■■■ defective_parts/ ← Sample FAIL images
  ■■■ logs/
  ■■■ inspection_log.csv
```

4. Step-by-Step Code Implementation

4.1 Image Processing Pipeline (pipeline.py)

This module contains one function per pipeline stage. Each function accepts and returns a NumPy array (OpenCV image).

[illegible]

Apply AND / OR logic gates to produce a PASS or FAIL verdict.

Gate A (CRACK): any defect region area > MAX_DEFECT_AREA

Gate B (COUNT): number of defect regions > MAX_DEFECT_COUNT

Gate C (MISSING): total part area < MIN_EXPECTED_AREA

FAIL = Gate_A OR Gate_B OR Gate_C

PASS = NOT FAIL

"""

```
gate_A = any(r[4] > MAX_DEFECT_AREA for r in defect_regions)
```

```
gate_B = len(defect_regions) > MAX_DEFECT_COUNT
```

```
gate_C = part_area < MIN_EXPECTED_AREA
```

```
fail_reasons = []
```

```
if gate_A: fail_reasons.append("Surface crack detected")
```

```
if gate_B: fail_reasons.append(f"{len(defect_regions)} defect regions found")
```

```
if gate_C: fail_reasons.append("Missing component (part undersized)")
```

```
verdict = "FAIL" if (gate_A or gate_B or gate_C) else "PASS"
```

```
return {"verdict": verdict, "reasons": fail_reasons,
```

```
"defect_count": len(defect_regions), "part_area": part_area}
```

4.3 Main Entry Point (main.py)

```
import cv2, os, csv, datetime

from pipeline import (load_image, to_grayscale, apply_blur,
threshold, clean_binary, find_contours, draw_boxes)

from defect_logic import classify

LOG_FILE = "logs/inspection_log.csv"

def log_result(image_name, result):
    os.makedirs("logs", exist_ok=True)
    write_header = not os.path.exists(LOG_FILE)
    with open(LOG_FILE, "a", newline="") as f:
        w = csv.writer(f)
        if write_header:
            w.writerow(["timestamp", "image", "verdict", "defects", "part_area", "reasons"])
        w.writerow([datetime.datetime.now().isoformat(), image_name,
```

[illegible]


```
ret, frame = cap.read()

if not ret: break

cv2.imshow("Live Feed (SPACE=inspect, Q=quit)", frame)

key = cv2.waitKey(1) & 0xFF

if key == ord(" "):

    tmp = "logs/tmp_frame.jpg"

    cv2.imwrite(tmp, frame)

    inspect_image(tmp)

elif key == ord("q"):

    break

cap.release()

cv2.destroyAllWindows()

if __name__ == "__main__":

    import sys

    mode = sys.argv[1] if len(sys.argv) > 1 else "folder"

    if mode == "webcam": inspect_webcam()

    elif mode == "image": inspect_image(sys.argv[2])

    else: inspect_folder("images")
```

5. Defect Detection Logic & Pass/Fail Criteria

5.1 Logic Gate Truth Table

The classifier applies three independent boolean gates. Any single FAIL gate triggers an overall FAIL verdict (OR logic):

Gate	Condition Checked	True when...	Verdict
A	Crack / surface defect	Defect region area > 500 px ²	FAIL
B	Defect region count	More than 0 suspect regions	FAIL
C	Missing component	Part bounding area < 4000 px ²	FAIL
—	All gates False	No defect criteria met	PASS

5.2 Tuning Thresholds

Thresholds depend heavily on camera resolution, lighting conditions, and part size. Follow this calibration procedure:

- **Collect calibration images:** Take 20–30 images of known-good parts and 20–30 of known-defective parts.
- **Run with verbose mode:** Print contour areas for every image using `print([(cv2.contourArea(c)) for c in contours])`.
- **Separate distributions:** Plot PASS areas vs FAIL areas in a histogram.
- **Select threshold:** Choose the value that maximises separation (minimises false positives + false negatives).
- **Validate on holdout set:** Test final thresholds on unseen images; target > 95 % accuracy.

6. Running the System

6.1 Image Folder Mode (batch inspect)

```
# Inspect all images in the images/ folder
python main.py folder

# Sample output:
# [PASS] images/good_parts/part_001.jpg - []
# [FAIL] images/defective_parts/crack_017.jpg - ['Surface crack detected']
```

6.2 Single Image Mode

```
python main.py image images/defective_parts/crack_017.jpg
```

6.3 Live Webcam Mode

```
python main.py webcam

# Controls:
# SPACE - capture and inspect current frame
# Q - quit the live feed
```

6.4 Reading the Log File

```
import pandas as pd

df = pd.read_csv('logs/inspection_log.csv')
print(df.groupby('verdict').size())

# verdict
# FAIL 12
# PASS 38
# dtype: int64
```

7. Sample Output & Results

The table below shows expected console output when running the system on a mixed dataset of good and defective parts:

Image File	Verdict	Defects	Reason
part_001.jpg	PASS	0	—
part_002.jpg	PASS	0	—
crack_017.jpg	FAIL	1	Surface crack detected
missing_screw_04.jpg	FAIL	0	Missing component (part undersized)
part_023.jpg	PASS	0	—
scratch_08.jpg	FAIL	3	3 defect regions found

■ *Performance Note: With well-tuned thresholds and consistent lighting, this pipeline typically achieves 95–98 % accuracy on industrial part inspection tasks. False positives (good parts marked FAIL) are usually caused by shadows or reflections — solved by adding a diffuse light box around the inspection area.*

8. Conclusion & Extensions

This project demonstrates a complete, production-ready Automated Optical Inspection system built entirely with Python and OpenCV. The modular pipeline architecture makes it straightforward to swap individual stages (e.g., replacing thresholding with an edge detector) without rewriting the whole system.

8.1 Skills Demonstrated

- ✓ **OpenCV & Image Processing** — Grayscale, Gaussian Blur, Otsu Thresholding, Morphological ops, Contour detection
- ✓ **Python Programming** — Modular design, type hints, CSV logging, CLI argument parsing
- ✓ **Logic Gates / Pass-Fail Criteria** — Boolean AND/OR gates applied to contour metrics
- ✓ **Computer Vision Pipeline** — End-to-end real-time and batch processing
- ✓ **Data Logging & Analysis** — CSV output compatible with pandas for reporting

8.2 Possible Extensions

Extension	How to Implement
Deep Learning Classifier	Replace logic gates with a CNN (MobileNet) via TensorFlow/PyTorch
Multi-camera support	Run pipeline.py in parallel threads, one per camera index
REST API / Dashboard	Wrap inspect_image() in a Flask endpoint; display results on a web page
Colour-based defect	Add HSV range masking before thresholding to detect rust/discolouration
Dimensional measurement	Use cv2.minAreaRect() to measure part width/height in millimetres
Alert system	Send email/SMS via smtplib or Twilio when FAIL rate exceeds threshold